

智能体辅助研发

以 AI Framework 为协作底座的 AI Coding 试点与推广方案

框架定位	AI Framework（智能体协作与研发治理框架）
试点范围	1 名客户端工程师 + 1 名后端工程师；4 周试点

一、背景与目标

为提升研发交付效率、减少重复性工作，并让代码、知识和决策能够持续沉淀，计划在部门内推进智能体辅助研发（Agentic Coding）工作方式。

本次推行不以“模型替代研发”为目标，而是以 AI Framework 为统一协作底座，通过产品规划、项目管理、工程实现三类智能体的分工协同，承担需求理解、任务拆解、代码实现、测试补全、问题定位、代码审查和文档沉淀等工作，使研发人员将更多精力投入架构判断、业务取舍、质量把控和关键决策。

同时，将工程规范、架构决策、领域规则、历史问题和任务记录沉淀至知识库，并通过 LDP / Sprint / Task / Gate 工作流，将需求、决策、实现、验证和验收串联为可追溯闭环，降低人员或会话切换带来的重复理解成本。

本阶段重点目标如下：

- 建立产品规划、项目管理、工程实现协同的智能体工作体系；
- 在不降低测试、Code Review 和发布门禁要求的前提下，提升常规研发任务的交付效率；
- 将项目知识、任务模板和实践经验沉淀为可复用资产；
- 通过小范围试点验证模型成本、实际产出和质量收益，为后续部门级推广提供依据。

二、已使用的智能体及能力分工

目前，AI Coding 体系以 AI Framework 为统一协作底座，采用“产品规划—项目管理—工程实现”三类智能体角色协同，并通过知识库、任务流程和验收机制保障交付质量。

. 产品经理智能体

主要负责需求澄清、方案分析、范围定义、风险识别和决策准备。

- 梳理业务需求、用户场景和非功能要求；
- 输出方案对比、关键取舍、风险点和待确认事项；
- 维护本地决策包（LDP）及 Sprint 的目标、范围和验收标准；
-

在涉及架构、数据、安全、外部接口等重要变更时，推动“问题确认—方案评审—决策记录—任务更新”的闭环。

不直接实施生产代码，重点是将模糊需求转化为边界清晰、可审查、可执行的决策和目标。

. 项目经理智能体

主要负责将已确认的目标拆解为可执行任务，并推进协作、验收和交付收口。

- 将 Sprint 拆分为具备明确 Owner、依赖、验收标准和测试要求的 Task；
- 协调多任务并行，提前识别文件、模块和职责冲突；
- 跟踪任务状态、依赖阻塞、验收结果和集成 Gate；
- 汇总进展、风险及需要人工决策的事项，形成可追溯的项目记录。

不替代项目负责人决策，而是将协作过程结构化，减少任务遗漏、反复沟通和上下文断裂。

. 高级软件工程师智能体

主要负责在已授权任务范围内完成代码实现、测试、提交和问题修复。

- 阅读最小必要上下文，完成代码开发、重构、测试补充和配置调整；
- 执行模块测试、集成测试及高风险变更的定向验证；
- 输出边界清晰的提交、验证结果和待验收交付；
- 识别生产装配、环境依赖、数据权限等隐性风险，并将超出任务边界的问题反馈为 Issue。

不绕过 Code Review、测试和发布门禁。涉及架构、数据库、权限或生产环境的变更，仍由工程师完成确认与最终决策。

. 知识库与工作流能力

AI Framework 不仅提供单次对话能力，还持续沉淀项目知识和工程上下文：

- 结构化保存工程规范、架构决策、领域规则、任务记录、问题复盘和项目背景；
- 支持共享规则与项目个性化知识分离维护、按需同步和持续积累；
- 通过 LDP / Sprint / Task / Gate 工作流，将需求、决策、实现、验证和验收串联；
- 让智能体在任务开始前按需加载相关规范、历史决策和任务信息，减少人员或会话切换后的重复理解成本。

目前，该体系已在智能体图片服务、日历日程代码梳理与缺陷修复、自动化部署等项目中落地复用。

三、第一阶段试点方案

第一阶段采取“小范围、可量化、可复盘”的双人试点，周期为 4 周。

试点角色	人数	AI Coding 配置	主要覆盖场景
客户端工程师	1 人	Codex Pro 5x	页面与组件开发、接口联调、常规 Bug 修复、测试补全
后端工程师	1 人	Codex Pro 20x	服务开发、跨仓库排障、复杂重构、代码审查、部署与持续迭代

试点期间，AI Framework 的产品、项目和高级软件工程师角色将作为统一工作方式使用；客户端和后端工程师则分别作为实际业务场景中的使用者与验收责任人。

试点遵循以下原则：

- 智能体负责辅助研发，工程师对代码质量和上线结果负责；
- 所有代码仍须通过既有测试、Code Review 和发布门禁；
- 每周复盘典型任务，记录任务耗时、模型使用量、返工情况和交付质量；
- 不以调用次数作为评价标准，而以交付效率、质量稳定性和成本收益作为核心指标。

四、Pro 5x 与 Pro 20x 的配置说明

项目	Codex Pro 5x	Codex Pro 20x
相对 Plus 的可用量	约 5 倍	约 20 倍
适用对象	日常高频使用 AI Coding 的工程师	每日高强度使用智能体的核心研发人员
典型工作	页面开发、组件改造、接口联调、常规 Bug 修复、测试补全	跨仓库任务、复杂排障、长上下文分析、大规模重构、连续 Agent 任务与代码审查
试点分配	客户端工程师	后端工程师
单人月预算	\$100，约 ¥720	\$200，约 ¥1,440

5x 与 20x 的差异主要在于可持续处理高强度任务的用量，而非模型能力不同。客户端工作通常集中在局部页面、组件、接口联调和缺陷修复，任务链路较短，配置 5x 能满足日常高频使用；后端工作涉及服务依赖、数据链路、日志排障、跨仓库分析及持续 Agent 任务，对上下文与任务额度要求更高，因此配置 20x。

五、模型选择与预算

第一阶段采用“托管模型为主、自建模型验证为辅”的策略。

1. 托管模型配置

方案	定位	单人月费用	建议
ChatGPT Plus / Codex	轻量编码协作、局部改动、日常问答和文档	\$20，约 ¥144	不建议作为高强度主力
Codex Pro 5x	客户端日常高频 AI Coding	\$100，约 ¥720	第一阶段客户端配置
Codex Pro 20x	后端复杂任务和持续 Agent Coding	\$200，约 ¥1,440	第一阶段后端配置
Claude Pro	代码分析、长文档理解、方案讨论	\$20，约 ¥144	可作为同级备选
Claude Max 5x / 20x	高频复杂开发、长上下文协作	\$100 / \$200	如需对照验证，可替代对应 Codex 席位

方案	定位	单人月费用	建议
Google AI Pro (Gemini)	Google Workspace、文档协作和轻量开发任务	\$19.99, 约 ¥144	团队重度使用 Google 生态时再考虑

第一阶段推荐预算如下：

预算项	月预算
客户端工程师：Codex Pro 5x	约 ¥720
后端工程师：Codex Pro 20x	约 ¥1,440
API / 超额额度弹性预留	¥500 – 1,000
合计	约 ¥2,660 – 3,160

第一阶段以 Codex 作为统一主力工具。Claude 或 Gemini 仅在需要做模型效果对照时按需引入，避免多个模型并行订阅却无法形成统一的方法、知识和任务模板。

2. 自建 Qwen3.6 验证

自建 Qwen3.6 的主要价值是私有数据可控、可接入内部代码库与知识库，并支持相对固定的流程类任务。第一阶段不以其替代主力 Coding 模型，而是验证私有化能力的可用性与投入产出。

推荐验证方向：

- 模型：优先验证 Qwen3.6-27B，或量化后的 Qwen3.6-35B-A3B；
- 场景：内部代码检索、知识库问答、项目规范查询、固定流程辅助任务；
- 原则：先验证质量、安全边界和实际使用频率，再决定是否扩大 GPU 投入。

部署方式	适用情况	月度成本估算	说明
云端按需 GPU	工作时段调用、快速验证	¥2,000 – 4,000	按工作日、工作时段估算
云端 24×7 常驻	需随时调用、接入内部服务	¥6,000 – 10,000	含 GPU、存储及基础网络预留，不含研发人力
自购单机服务器	预计持续使用 1 年以上	首次 ¥45,000 – 70,000；折算约 ¥2,000 – 4,000/月	按 36 个月折旧，含基础运维预留

Qwen3.6-35B-A3B 的原始权重约 71.9GB，官方的大上下文服务示例采用多卡并行。因此，双人试点应优先采用量化模型，并限制上下文长度和并发；若需要全精度、大上下文和多并发，硬件成本会明显上升。

六、部门推广计划

阶段一. 双人试点（4 周）

- 人员：1 名客户端工程师、1 名后端工程师；

- 配置：客户端使用 Codex Pro 5x，后端使用 Codex Pro 20x；
- 方法：统一接入 AI Framework 的角色、知识库和工作流；
- 交付：每周沉淀需求拆解、代码实现、测试补全、代码审查和故障定位等典型任务模板；
- 衡量指标：任务交付周期、返工率、代码审查问题数、测试补全量、实际人工耗时、模型成本；
- 验收目标：在质量门禁不降低的前提下，验证常规研发任务效率提升效果，并形成可量化的基线对比。

阶段二. 小组扩展（6–8 周）

- 将试点中已验证有效的提示词、任务模板、角色边界和知识库接入方式固化为团队规范；
- 扩展至客户端、后端、测试和产品协作链路；
- 建立任务台账，持续记录模型使用量、成本、产出和失败案例；
- 基于内部代码与知识库场景，评估 Qwen3.6 私有化部署的实际收益。

阶段三. 部门推广与平台化

- 将 AI Framework 接入研发流程，形成“需求—决策—任务—开发—验收—复盘”的可追溯闭环；
- 建立模型路由策略：复杂开发任务使用高强度托管模型，敏感资料和内部知识检索优先走自建模型；
- 建立用量预算、权限控制、数据边界、质量门禁和效果看板；
- 依据真实使用数据，决定扩大托管席位或增加私有化 GPU 资源。

七、结论

建议以“1 名客户端 + Codex Pro 5x、1 名后端 + Codex Pro 20x”的方式启动 4 周试点，月度预算控制在 ¥2,660–3,160。

试点以 AI Framework（智能体协作与研发治理框架）统一智能体角色、知识库和研发工作流，重点验证真实研发任务的效率、质量与成本。自建 Qwen3.6

同期仅进行私有化能力验证；待形成稳定的使用数据后，再决定是否扩大部署和进行部门级推广。

附. 价格与技术口径说明

文中美元换算按 1 美元≈7.2 元人民币估算，实际结算受汇率、税费和地区可用性影响。Codex 的计划、用量和超额规则以官方购买页为准；Qwen3.6 的模型版本与部署方式以官方开源仓库为准。